

Table of Contents

AI SOLUTIONS ARCHITECT · MODULO 06 · AI SERVICE DEPLOYMENT

Servizio di stima dei tempi di consegna

Caso LogiFast Solutions

Simone La Porta · 2026-07-01

Repository pubblico (codice Flask eseguibile, notebook con plot e metriche renderizzati da GitHub, modello `delivery.pkl`, test):

https://github.com/Shurtug4l/AI_solutions_architect_course/tree/master/06_AI_services_deployment/_PRJ_logifast_delivery_time_service

Questo documento e' la sintesi autosufficiente del capstone per la valutazione; il codice completo e il notebook eseguito sono nel repository sopra.

Report finale - Servizio di stima dei tempi di consegna

Cliente: LogiFast Solutions (fornitore logistico urbano e interurbano). **Modulo 06 - AI Service Deployment.** Autore: Simone La Porta.

1. Contesto e valore

LogiFast vuole stimare il tempo di consegna (time-to-delivery) di ogni ordine al momento della presa in carico, per informare clienti e operatori, pianificare le rotte e ridurre reclami e reinvii. La soluzione espone un modello predittivo via API REST in Flask, consumabile dal gestionale e dal front-end e-commerce.

Il valore non sta solo nella stima puntuale ma nell'**affidabilita' del servizio nel tempo**: versionamento e monitoraggio permettono di accorgersi quando il modello smette di essere accurato e di intervenire, invece di scoprirlo dai reclami.

2. Dati

Input noti alla presa in carico: origine, destinazione, data/ora di ritiro, peso, tipo di servizio. Output: stima in ore, con score di affidabilita' e banda indicativa. Lo schema completo e i vocabolari (20 citta', servizi Express/Premium) sono in `docs/data_schema.md`.

Un primo risultato dell'esplorazione e' una **discrepanza tra traccia e artefatto**: la traccia elenca cinque campi di input, ma il modello fornito ne usa quattro - `pickup_datetime` non entra nella predizione. L'API lo accetta comunque (contratto stabile in vista del riaddestramento) e lo dichiara come non usato.

3. Analisi esplorativa e validazione

Il dettaglio è in notebooks/exploration_validation.ipynb. L'artefatto è una Pipeline scikit-learn: OneHotEncoder sulle categoriche (città, servizio) + passthrough sul peso, seguito da una LinearRegression.

Due letture, una intrinseca e una su dati.

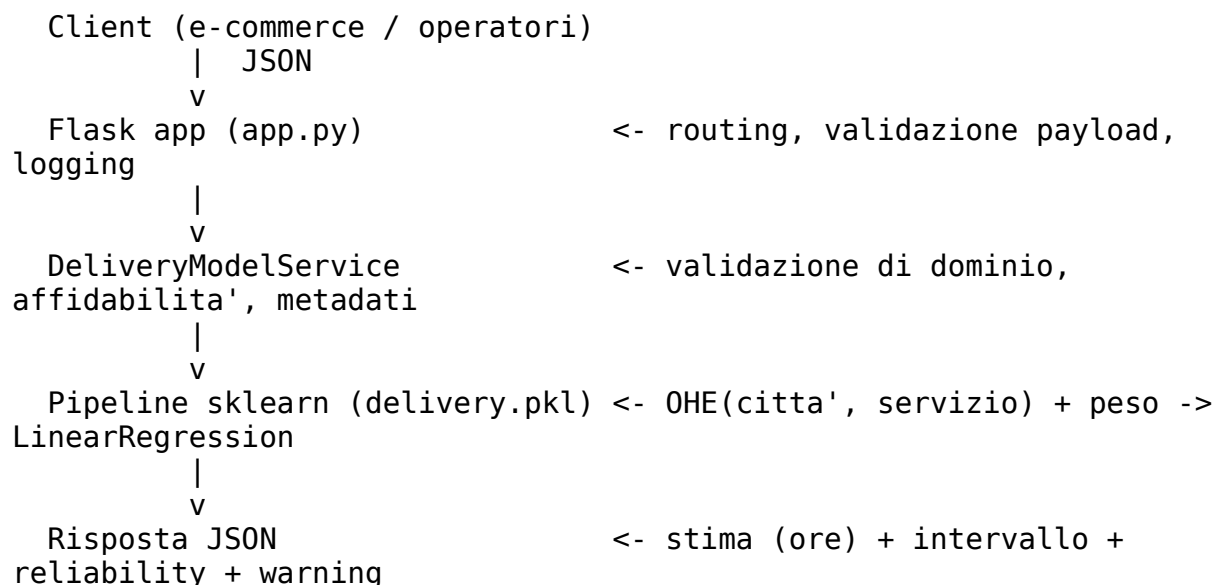
- **Intrinseca** (solo coefficienti e risposte del modello): il modello è poco sensibile agli input. Il peso da 1 a 50 kg sposta la stima di ~2 ore, Express e Premium di mezz'ora, le destinazioni restano in una banda stretta attorno alle 47 ore. Una regressione lineare su one-hot di città cattura un effetto additivo per città, **non la distanza** tra origine e destinazione.
- **Su dati sintetici** (in assenza di uno storico etichettato, dataset generato da un processo documentato che incorpora la distanza geografica reale tra le città): MAE 20.2 ore, RMSE 22.5 ore, **R2 = -4.82**. Un R2 negativo significa che il modello fa peggio di una semplice media: predice ~costante mentre il tempo reale varia molto con la distanza.

La conclusione non è "il modello è inutile": la traccia chiede di deployarlo, ed è quello che facciamo. La conclusione è che il suo limite è noto, misurato e sistematico (errore sui tragitti lunghi), quindi **gestibile** dal monitoraggio invece che subito.

4. Scelta delle metriche

MAE e RMSE perché il problema è una regressione e l'errore va letto nell'unità di business (ore): MAE per l'errore tipico, RMSE perché penalizza di più gli errori grossi, che qui sono concentrati sui tragitti lunghi. R2 per posizionare il modello rispetto a una baseline (la media): è la metrica che ha reso evidente il problema. In produzione la metrica chiave diventa l'**errore per segmento** (tratta, peso, servizio): la media aggregata nasconderebbe proprio il punto debole.

5. Architettura della soluzione



Due piani separati di proposito: trasporto HTTP (`app.py`) e contratto del modello (`model_service.py`). La logica di predizione e' testabile senza server e il modello non dipende da Flask. Endpoint: `POST /predict`, `POST /predict/batch`, `GET /health`, `GET /model`. Dettaglio in `docs/api_spec.md`.

Scelte difese altrove e qui sintetizzate: `pickup_datetime` accettato ma segnalato come non usato; citta' fuori vocabolario gestite con warning e affidabilita' ridotta invece che con un errore; banda dell'intervallo pari all'RMSE sintetico (~22.5 ore), ampia perche' onesta sul modello debole; sicurezza del pickle (verifica statica + hash SHA-256).

6. Elementi MLOps

Versionamento (dataset, modello, model card), pipeline concettuale di automazione, monitoraggio (logging gia' attivo, metriche runtime e di qualita' per segmento, drift, alerting) e governance (rollback su artefatti versionati, trigger di riaddestramento) sono progettati in `docs/mlops_design.md`. Il prossimo ciclo di training ha un candidato ovvio: aggiungere distanza e `pickup_datetime`, le informazioni che il modello attuale ignora.

7. Piano di rollout

Promozione a stadi, ciascuno con criterio di avanzamento e di rollback.

1. **Staging**: la suite `tests/test_api.py` passa, l'immagine Docker si avvia, `/health` e `/model` rispondono.
2. **Shadow**: il servizio gira in parallelo alla procedura attuale, le predizioni sono loggate ma non mostrate. Si raccoglie ground truth e si misura l'errore per segmento.
3. **Canary**: una quota ridotta di traffico reale (es. 5-10%). Avanzamento se latenza e errore restano nei limiti; rollback se un alert critico scatta.
4. **Full**: rollout completo, con monitoraggio e trigger di riaddestramento attivi.

8. Limiti e sviluppi

- Il modello ignora distanza e data/ora: e' il limite dominante, gia' misurato. Sviluppo prioritario: riaddestrare includendole.
- L'unita' (ore) e' un'assunzione sul range osservato, da confermare con la documentazione di training.
- `reliability_score` e intervallo sono proxy dichiarati, non statistiche rigorose: con dati reali si possono sostituire con intervalli di predizione veri.
- La validazione e' su dati sintetici: serve uno storico etichettato per metriche di produzione affidabili.

Appendice A - Schema dei dati

Input (record di ordine)

Campi accettati in `POST /predict` e in ogni elemento di `POST /predict/batch`.

Campo	Tipo	Obbligatorio	Vincoli	Usato dal modello
pickup_location	stringa	si	città di origine	si (one-hot)
delivery_location	stringa	si	città di destinazione	si (one-hot)
pickup_datetime	stringa	si	ISO 8601 consigliato	no (vedi nota)
weight	numero	si	> 0, in kg	si (passthrough)
service_type	stringa	si	Express o Premium	si (one-hot)

Nota su pickup_datetime. Il modello fornito è addestrato su quattro feature e non usa la data/ora di ritiro. Il campo resta nel contratto per coerenza con la specifica e per non rompere i client quando il modello verrà riaddestrato includendolo. La validazione è lasca: si tenta il parse ISO 8601 e si avvisa, senza bloccare. L'endpoint `/model` lo elenca in `unused_fields`.

Vocabolari chiusi

- **service_type:** Express, Premium. Un valore fuori da questo insieme produce un errore 400: è un insieme di business chiuso, non ha senso degradare silenziosamente.
- **pickup_location / delivery_location:** 20 città note, apprese dall'artefatto e esposte da `/model`: Ancona, Bari, Bologna, Cagliari, Catania, Firenze, Genova, Lecce, Milano, Napoli, Palermo, Perugia, Pescara, Reggio Calabria, Roma, Salerno, Sassari, Torino, Trapani, Verona. Una città fuori vocabolario **non** genera errore (l'OneHotEncoder usa `handle_unknown='ignore'`, la codifica a zeri) ma abbassa il `reliability_score` e aggiunge un warning. La scelta è deliberata: una stima silenziosamente degradata è peggio di una segnalata.

Output (predizione)

Campo	Tipo	Descrizione
predicted_delivery_time_hours	numero	stima del tempo di consegna, in ore
confidence_interval_hours	[numero, numero]	banda indicativa (~RMSE sintetico, ~22.5 h), non statistica
reliability_score	numero in [0, 1]	euristica di in-distribution, non una probabilità
unit	stringa	ore
model_version	stringa	versione dell'artefatto deployato
warnings	lista di stringhe	anomalie non bloccanti
input	oggetto	record validato (solo in <code>/predict</code>)

Unita' di misura. Il modello restituisce un numero senza unita'. Sul range osservato (circa 39-55, media ~46) l'interpretazione coerente con consegne interurbane e' in ore (~2 giorni medi), non minuti. Assunzione dichiarata, da confermare con la documentazione di training del modello.

reliability_score. Parte da 1.0 e viene moltiplicato per 0.6 per ogni citta' fuori vocabolario e per 0.9 se il peso supera i 100 kg. Non e' una probabilita': una `LinearRegression` non espone `predict_proba`. E' un proxy di quanto l'input cade nella distribuzione su cui il modello e' affidabile.

Appendice B - Specifica API REST

Base URL (sviluppo): `http://localhost:8080`. Tutte le richieste e risposte sono in `application/json`. Lo schema dei campi e' in `data_schema.md`.

POST /predict

Predizione su un singolo ordine.

Richiesta

```
{
  "pickup_location": "Milano",
  "delivery_location": "Roma",
  "pickup_datetime": "2026-07-01T09:30:00",
  "weight": 5.0,
  "service_type": "Express"
}
```

Risposta 200

```
{
  "predicted_delivery_time_hours": 43.07,
  "confidence_interval_hours": [20.57, 65.57],
  "reliability_score": 1.0,
  "unit": "ore",
  "model_version": "1.0.0",
  "warnings": [],
  "input": { "pickup_location": "Milano", "delivery_location": "Roma",
    "pickup_datetime": "2026-07-01T09:30:00", "weight": 5.0,
    "service_type": "Express" }
}
```

Con input fuori distribuzione la risposta resta 200 ma `reliability_score` scende e `warnings` si popola (es. citta' ignota, `pickup_datetime` non parsabile).

POST /predict/batch

Predizione su una lista di ordini. Un record malformato non fa cadere il batch: viene isolato come errore, gli altri vengono predetti.

Richiesta: lista di record con lo stesso schema di /predict.

Risposta 200

```
{
  "count": 2,
  "results": [
    { "index": 0, "status": "ok", "predicted_delivery_time_hours":
55.19,
      "confidence_interval_hours": [32.69, 77.69],
      "reliability_score": 1.0,
      "unit": "ore", "model_version": "1.0.0", "warnings": [] },
    { "index": 1, "status": "error", "errors": ["weight deve essere >
0"] }
  ]
}
```

GET /health

Stato operativo del servizio (readiness). Il campo status vale OK con codice 200 quando il modello risulta caricato, ERROR con 503 quando l'artefatto non e' disponibile: cosi' il servizio non riceve traffico prima di essere pronto.

```
{ "status": "OK", "timestamp": "2026-07-01T07:49:00.152448+00:00" }
```

GET /model

Versione e contratto del modello: utile per i client e per il versionamento.

```
{
  "model_version": "1.0.0",
  "model_type": "LinearRegression",
  "framework": "scikit-learn 1.6.1",
  "pinned_sklearn": "1.6.1",
  "features_used": ["pickup_location", "delivery_location", "weight",
"service_type"],
  "request_fields": ["pickup_location", "delivery_location",
"pickup_datetime", "weight", "service_type"],
  "unused_fields": ["pickup_datetime"],
  "output_unit": "ore",
  "known_locations": ["Ancona", "... 20 citta ..."],
  "known_service_types": ["Express", "Premium"],
  "artifact_sha256": "cb29574f...",
  "loaded_at": "2026-06-30T21:49:00+00:00"
}
```

Formato degli errori

Gli errori bloccanti hanno sempre la stessa forma, con la lista puntuale dei problemi:

```
{ "status": "error", "errors": ["campi mancanti: weight"] }
```

Codici di stato

Codice	Quando
200	richiesta valida (anche con warning per input degradato)
400	payload assente/non-JSON, campi mancanti, <code>service_type</code> non valido, body batch non lista
404	endpoint inesistente
500	errore interno non previsto (loggato lato server)
503	modello non disponibile (/health risponde ERROR, /predict e /predict/batch rifiutano)

Appendice C - Disegno MLOps

Come da traccia, questi elementi sono **progettati e documentati**, non implementati come ecosistema reale. Dove esiste già un gancio nel codice (logging, hash dell'artefatto, endpoint /model, validazione con warning) lo richiamo: il disegno non parte da zero, estende quello che il servizio espone.

Il filo conduttore è una scoperta della validazione (vedi notebooks/): il modello fornito **ignora la distanza** tra origine e destinazione e predice un valore quasi costante (R^2 negativo, -4.82, contro una realtà distanza-aware). Un buon disegno MLOps deve rendere questo degrado **visibile e azionabile**, non nascondere.

1. Versionamento

Dataset

Ogni dataset di training è uno snapshot immutabile, identificato da `dataset_id` (es. `delivery-2026-06`) e da un hash del contenuto. Si versionano: la finestra temporale, lo schema, le 20 città in vocabolario, le statistiche di base (distribuzioni di peso, tratte, servizio). In assenza di un data lake, anche un semplice manifest versionato (CSV + `manifest.json` con hash) è sufficiente a garantire riproducibilità.

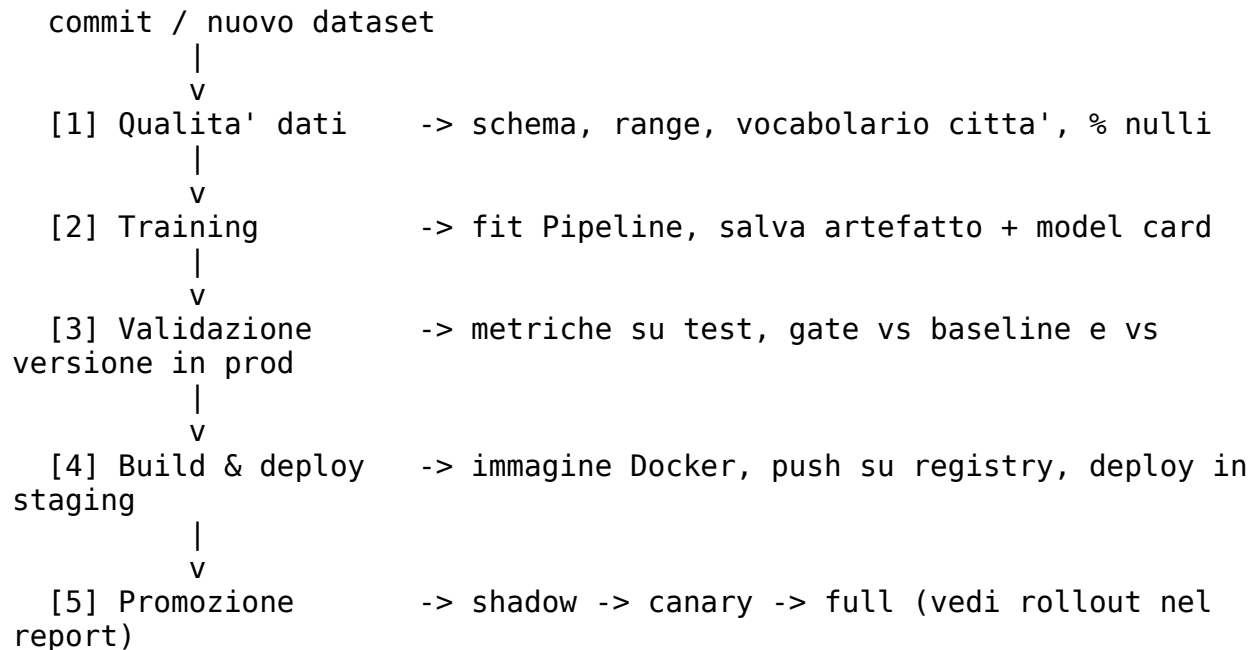
Modello e metadati

L'artefatto è versionato con SemVer (`MODEL_VERSION`, oggi `1.0.0`) e accompagnato da una **model card** con: tipo (`LinearRegression`), feature usate, `dataset_id` di training, metriche di validazione, data, commit Git, e l'hash SHA-256 dell'artefatto (già calcolato al load in `model_service.py`). L'endpoint /model espone questi metadati a runtime.

Compatibilita' del contratto

Il contratto API (`request_fields`) e' piu' ampio delle feature del modello: include `pickup_datetime`, oggi inutilizzato. Questo e' deliberato e va versionato: quando il modello verra' riaddestrato includendo la data/ora, i client esistenti non cambieranno. Una modifica che rimuovesse o rinominasse un campo richiesto sarebbe invece breaking e imporrebbe un bump di major.

2. Automazione (pipeline concettuale)



[1] Qualita' dei dati

Controlli automatici prima del training: schema atteso, `weight > 0`, `service_type` nel vocabolario, citta' note, percentuale di valori mancanti sotto soglia. Un fallimento blocca la pipeline: meglio non addestrare che addestrare su dati sporchi.

[2] Training

Riproducibile: seed fisso, dipendenze pinnate (`requirements.txt`, `scikit-learn==1.6.1`), `dataset_id` registrato. L'output e' l'artefatto piu' la model card.

[3] Test e validazione pre-deploy

Due cancelli. Il primo, **test di regressione del modello**: le metriche sul set di test non devono peggiorare oltre una soglia rispetto alla versione in produzione (no regressioni silenziose). Il secondo, **test del servizio**: la suite `tests/test_api.py` deve passare sulla nuova immagine (endpoint, validazione, batch). Solo con entrambi verdi si procede.

[4] Build e deploy

Immagine Docker costruita dalla pipeline, taggata con `MODEL_VERSION` e commit, spinta su un registry. Il deploy in staging precede sempre la produzione.

3. Monitoraggio

Logging di richieste e predizioni

Gia' attivo: `app.py` logga per ogni richiesta la tratta, il servizio, la stima e il `reliability_score`. In produzione questi log alimentano un archivio strutturato (una riga per predizione: input, output, versione modello, timestamp) che e' la base di tutto il resto.

Metriche runtime

Latenza (p50/p95/p99), throughput, tasso di errori 4xx/5xx, quota di richieste con `reliability_score` basso. Sono le metriche di servizio, indipendenti dalla qualita' del modello.

Metriche di qualita'

Richiedono il **ground truth differito**: il tempo di consegna effettivo, noto solo dopo la consegna. Quando arriva, si accoppia alla predizione loggata e si calcola l'errore (MAE, RMSE) **per segmento**: per tratta, fascia di peso, servizio. Il monitoraggio per segmento non e' un dettaglio: la validazione ha mostrato che l'errore di questo modello e' concentrato sui tragitti lunghi, e una media aggregata lo nasconderebbe.

Drift detection

Due piani. **Data drift**: la distribuzione degli input cambia rispetto al training (nuove citta', pesi fuori range, mix di servizi diverso) - intercettabile gia' dai warning e dal `reliability_score`, aggregati nel tempo. **Concept drift**: la relazione input-tempo cambia (nuovi corrieri, congestione) - visibile come deriva dell'errore per segmento.

Alerting

Soglie su: errore per segmento oltre una banda (es. $RMSE > 1.5 \times \text{baseline}$ su una tratta), quota di predizioni a bassa affidabilita' in crescita, comparsa stabile di citta' fuori vocabolario, p95 di latenza oltre l'SLA. L>alert e' azionabile: indica il segmento, non solo "il modello peggiora".

4. Governance

Rollback

Gli artefatti sono versionati e immutabili: il rollback e' ridistribuire l'immagine con la `MODEL_VERSION` precedente. L'endpoint `/model` e l'hash SHA-256 permettono di verificare quale versione e' davvero in esecuzione. Criterio: si effettua rollback se la nuova versione viola un gate di qualita' in canary o se un alert critico scatta entro la finestra di osservazione.

Trigger di riaddestramento

- **Degrado:** errore per segmento oltre soglia su una finestra mobile.
- **Drift dei dati:** spostamento stabile della distribuzione degli input.
- **Nuovi dati:** disponibilit  di uno storico etichettato sufficiente (es. trimestrale).
- **Gap noto:** il candidato naturale del prossimo ciclo   includere la distanza tra citta' e la pickup_datetime, le due informazioni che il modello attuale ignora.

Tracciabilit  e audit

Ogni predizione   ricostruibile: input loggato, model_version, hash dell'artefatto, timestamp. Questo chiude il cerchio tra dato, modello e decisione, ed   il prerequisito per qualunque analisi a posteriori di un reclamo o di un errore sistematico.

Appendice D - Avvio rapido

Richiede Python 3.12 o superiore (numpy 2.5 non supporta la 3.11).

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt           # scikit-learn==1.6.1 pinnato
flask --app app run --port 8080          # oppure: python app.py
pytest -q                                # test di integrazione
```